

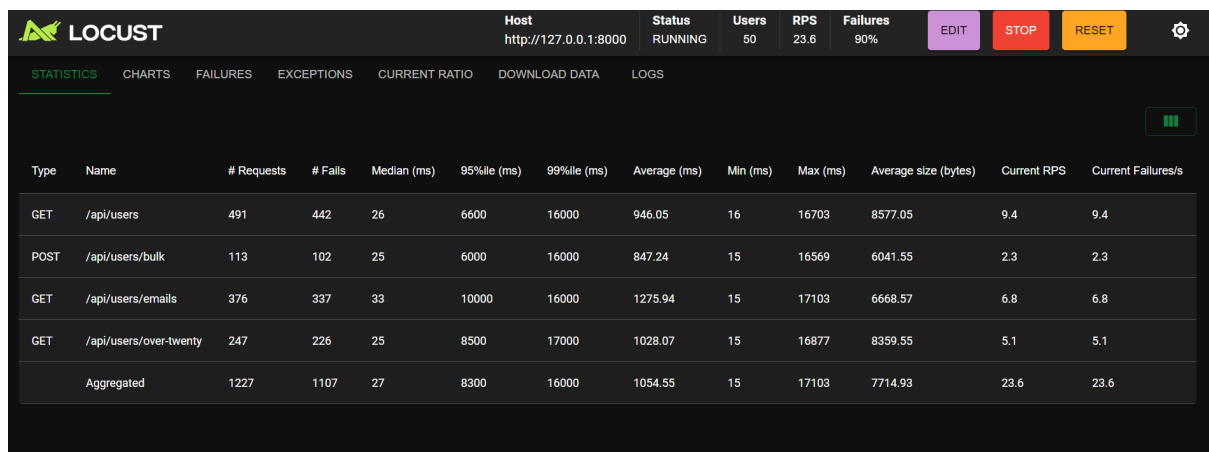
Taller Ingeniería de Software 2

1. Actualizar dependencias
composer install
2. Correr las migraciones con comando
php -d memory_limit=-1 artisan migrate:fresh --seed

Al presentar tanta información la memoria puede colapsar, este comando ayuda a delimitar la memoria ram.

3. Ejecutar los seeders para llevar la información a la BD llamada "laravel"
4. Testear el end point para ver si nos trae la información
5. Al haber tantos datos, el controlador UserController debe tener una paginación, se debe modificar que traiga todos los usuarios y solo 100 o los que se deseen.
6. Volver a testear el end point
7. Ahora si devuelve 100 usuarios gracias a la paginación
8. Ejecutar pip install locust
9. Direccionar las pruebas para la API que vamos a testear
locust -f locustfile.py --host=<http://127.0.0.1:8000>
10. Observamos los resultados abriendo en el navegador localhost:8089

Prueba de carga



Una observación es que mi equipo tiene instalado laravel desde el entorno de Herd, al hacer la prueba en mi equipo con 50 usuarios y 1 usuarios por segundo, presenta fallas luego de 90 peticiones. Al hacer la comparación con un compañero, el presenta luego de 2 minutos un % de error del 10% y mi prueba del 90% (La prueba desde mi equipo es mucho más rápida que en la de el)

El punto óptimo es éste **Usuarios: 10, Usuario por segundo: 1**

 LOCUST

Host

http://127.0.0.1:8000

Status

RUNNING

Users

10

RPS

2.7

Failures

18%

EDIT

LOADING

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/users	34	7	1100	2400	2800	1018.73	17	2777	22234.18	1	0
POST	/api/users/bulk	8	1	1600	2300	2300	1456.62	45	2347	1428.5	0.5	0
GET	/api/users/emails	22	3	1400	2200	2500	1305.28	18	2487	7070.73	0.8	0
GET	/api/users/over-twenty	9	2	1100	2200	2200	1114.54	32	2206	22562.44	0.4	0
Aggregated		73	13	1300	2300	2800	1164.89	17	2777	15424.77	2.7	0

Prueba de estrés.
Test: 200 usuarios, 5 usuarios por segundo

 LOCUST

Host

http://127.0.0.1:8000

Status

STOPPED

RPS

3.3

Failures

73%

NEW

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

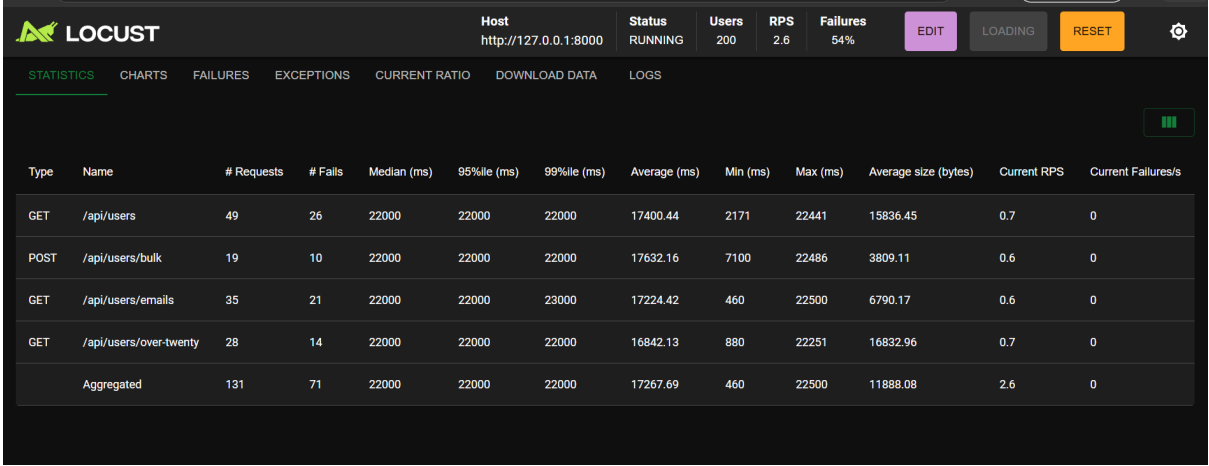


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/users	92	68	4700	13000	14000	5680.43	122	14331	11747.5	1	0.2
POST	/api/users/bulk	19	15	6600	14000	14000	5763.34	249	14143	5370.16	0.3	0
GET	/api/users/emails	68	45	6400	14000	14000	6400.53	140	13709	6772.1	1.5	0.3
GET	/api/users/over-twenty	40	31	5600	13000	14000	5917.28	289	13567	11226.4	0.5	0.1
	Aggregated	219	159	5500	13000	14000	5954.48	122	14331	9554.16	3.3	0.6

Se comporta bien hasta aproximadamente la petición número 200, aproximadamente 25 segundos.

Prueba de capacidad

Test: Usuarios: 200, Usuarios por segundo: 50



The screenshot shows the Locust web interface with a dark theme. At the top, the Locust logo is on the left, and a summary bar on the right displays: Host (http://127.0.0.1:8000), Status (RUNNING), Users (200), RPS (2.6), Failures (54%), and buttons for EDIT, LOADING, and RESET. Below the summary bar is a navigation menu with links: STATISTICS (active), CHARTS, FAILURES, EXCEPTIONS, CURRENT RATIO, DOWNLOAD DATA, and LOGS. A green 'STOP' button is on the right. The main area contains a table with test results.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/users	49	26	22000	22000	22000	17400.44	2171	22441	15836.45	0.7	0
POST	/api/users/bulk	19	10	22000	22000	22000	17632.16	7100	22486	3809.11	0.6	0
GET	/api/users/emails	35	21	22000	22000	23000	17224.42	460	22500	6790.17	0.6	0
GET	/api/users/over-twenty	28	14	22000	22000	22000	16842.13	880	22251	16832.96	0.7	0
Aggregated		131	71	22000	22000	22000	17267.69	460	22500	11888.08	2.6	0

Se comporta bien hasta la petición 131 aproximadamente, tiene un comportamiento estable por aproximadamente 15 segundos.

Conclusiones:

Impacto del entorno de ejecución (Laravel Herd vs. Servidor Tradicional): La diferencia en la rapidez de las pruebas y la alta tasa de fallos (90% frente al 10% de mi compañero) evidencia el comportamiento de Laravel Herd. Herd es capaz de procesar múltiples peticiones de forma simultánea y mucho más rápida que el servidor integrado. Sin embargo, esta rapidez satura de inmediato los recursos de tu máquina, provocando rechazos de conexión prematuros.

Eficacia de la paginación frente a volúmenes masivos: La imposibilidad de correr el sistema con el conjunto completo de 1.5 millones de registros demostró que devolver bases de datos completas en un solo arreglo JSON es una mala práctica crítica. La implementación de la paginación en el UserController fue indispensable para evitar el desbordamiento de memoria (Error 500) y hacer que la API fuera evaluable bajo tráfico.